

## Unit-4

### Introduction to Database System

#### Relational Algebra

Relational Algebra is a procedural language. In Relational Algebra, order is specified in which operations have to be performed. Basic operation in relational algebra are:

1. Select ( $\sigma$ )
2. Project ( $\Pi$ )
3. Union ( $\cup$ )
4. Set Difference ( $-$ )
5. Cartesian product ( $\times$ )

#### Relational Calculus

Relational Calculus is the formal query language. It is also known as **Declarative language**. In Relational Calculus, order is not specified in which operation has to be performed. Relational Calculus means what result we have to obtain. Relational Calculus has two variations:

1. Tuple Relational Calculus (TRC)
2. Domain Relational Calculus (DRC)

Relational Calculus is denoted as:

$\{ t \mid P(t) \}$     Where,

t: the set of tuples

p: is condition which is true for the given set of tuples.

## Difference between Relational Algebra and Relational Calculus:

| S.NO | Basis of Comparison       | Relational Algebra                                                                                                                                                        | Relational Calculus                                                                                                      |
|------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1.   | Language Type             | It is a Procedural language.                                                                                                                                              | Relational Calculus is a Declarative (non-procedural) language.                                                          |
| 2.   | Procedure                 | Relational Algebra means how to obtain the result.                                                                                                                        | Relational Calculus means what result we have to obtain.                                                                 |
| 3.   | Order                     | In Relational Algebra, the order is specified in which the operations have to be performed.                                                                               | In Relational Calculus, the order is not specified.                                                                      |
| 4.   | Domain                    | Relational Algebra is independent of the domain.                                                                                                                          | Relation Calculus can be domain-dependent                                                                                |
| 5.   | Programming language      | Relational Algebra is nearer to a programming language.                                                                                                                   | Relational Calculus is nearer to natural language.                                                                       |
| 6.   | Inclusion in SQL          | SQL includes some features from the relational algebra.                                                                                                                   | SQL is based to a greater extent on the tuple relational calculus.                                                       |
| 7.   | Relationally completeness | Relational Algebra is one of the languages in which queries can be expressed but the queries should also be expressed in relational calculus to be relationally complete. | For a database language to be relationally complete, the query written in it must be expressible in relational calculus. |
| 8.   | Query Evaluation          | The evaluation of the query relies on the order specification in which the operations must be performed.                                                                  | The order of operations does not matter in relational calculus for the evaluation of queries.                            |

| S.NO | Basis of Comparison | Relational Algebra                                                                                                                                                       | Relational Calculus                                                                                                                                                                                       |
|------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9.   | Database access     | For accessing the database, relational algebra provides a solution in terms of what is required and how to get that information by following a step-by-step description. | For accessing the database, relational calculus provides a solution in terms as simple as what is required and lets the system find the solution for that.                                                |
| 10.  | Expressiveness      | The expressiveness of any given language is judged using relational algebra operations as a standard.                                                                    | Completeness of a language is measured in the manner that it is least as powerful as calculus. That implies relation defined using some expression of calculus is also definable by some other expression |

## Functional Dependency

In relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute. It is denoted as  $X \rightarrow Y$ , where attribute set on the left side of the arrow, **X** is called **Determinant**, **Y** is called **Dependent**.

**Example:**

| roll_no | name | dept_name | dept_building |
|---------|------|-----------|---------------|
| 42      | abc  | CO        | A4            |
| 43      | pqr  | IT        | A3            |
| 44      | xyz  | CO        | A4            |

**From above table we can conclude some valid functional dependencies:**

- $\text{roll\_no} \rightarrow \{ \text{name}, \text{dept\_name}, \text{dept\_building} \}$ ,  $\rightarrow$   
Here  $\text{roll\_no}$  can determine values of  $\text{name}$ ,  $\text{dept\_name}$  and  $\text{dept\_building}$ , hence a valid Functional dependency
- $\text{roll\_no} \rightarrow \text{dept\_name}$  , Since,  $\text{roll\_no}$  can determine whole set of  $\{ \text{name}, \text{dept\_name}, \text{dept\_building} \}$ , it can determine its subset  $\text{dept\_name}$  also.
- $\text{dept\_name} \rightarrow \text{dept\_building}$  ,  $\text{Dept\_name}$  can identify the  $\text{dept\_building}$  accurately, since departments with different  $\text{dept\_name}$  will also have a different  $\text{dept\_building}$

**Here are some invalid functional dependencies:**

- $\text{name} \rightarrow \text{dept\_name}$  Students with the same name can have different  $\text{dept\_name}$ , hence this is not a valid functional dependency.
- $\text{dept\_building} \rightarrow \text{dept\_name}$  There can be multiple departments in the same building. Example, in the above table departments ME and EC are in the same building B2, hence  $\text{dept\_building} \rightarrow \text{dept\_name}$  is an invalid functional dependency.

## Types of Functional Dependencies in DBMS

### 1. Trivial Functional Dependency

In this, a dependent is always a subset of determinant. i.e. If  $X \rightarrow Y$  and  $Y$  is the subset of  $X$ , then it is called trivial functional dependency

**Example:**

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |
| 44      | xyz  | 18  |

Here,  $\{\text{roll\_no, name}\} \rightarrow \text{name}$  is a trivial functional dependency, since the dependent **name** is a subset of determinant set  **$\{\text{roll\_no, name}\}$** .

### 2. Non-trivial Functional Dependency

In this, the dependent is strictly not a subset of the determinant. i.e. If  $X \rightarrow Y$  and  $Y$  is not a subset of  $X$ , then it is called Non-trivial functional dependency.

Here  $\{\text{roll\_no, name}\} \rightarrow \text{age}$  is a non-trivial functional dependency, since age is not a subset of  **$\{\text{roll\_no, name}\}$**

### 3. Multivalued Functional Dependency

In this, entities of dependent set are not dependent on each other. If  $a \rightarrow \{b, c\}$  and there exists no functional

dependency between b and c then it is called a **multivalued functional dependency**.

Here, **roll\_no**  $\rightarrow$  {**name**, **age**} is a multivalued functional dependency, since the dependents name & age are not dependent on each other.

#### 4. Transitive Functional Dependency

In this, dependent is indirectly dependent on determinant. i.e. If **a**  $\rightarrow$  **b** & **b**  $\rightarrow$  **c**, then according to axiom of transitivity, **a**  $\rightarrow$  **c**.

**For example,**

| enrol_no | name | dept | building_no |
|----------|------|------|-------------|
| 42       | abc  | CO   | 4           |
| 43       | pqr  | EC   | 2           |
| 44       | xyz  | IT   | 1           |
| 45       | abc  | EC   | 2           |

Here, **enrol\_no**  $\rightarrow$  **dept** and **dept**  $\rightarrow$  **building\_no**. Hence, **enrol\_no**  $\rightarrow$  **building\_no** is an indirect functional dependency, called Transitive functional dependency.

#### 5. Fully Functional Dependency

In this, an attribute or a set of attributes uniquely determines another attribute or set of attributes. If an

attribute Q is fully functional dependent on another attribute P, if it is Functionally Dependent on P and not on any of the proper subset of P.

For ex.  $XY \rightarrow Z$  then Z will depend completely on XY.

## **6. Partial Functional Dependency**

In this, a non key attribute depends on a part of the composite key, rather than the whole key. If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then  $X \rightarrow Z$  is a partial functional dependency in RDBMS.

For ex.  $XY \rightarrow Z$  then Z can either depend on X or Y individually.

## **Advantages of Functional Dependencies**

### **1. Data Normalization**

Data normalization is the process of organizing data in a database in order to minimize redundancy and increase data integrity. Functional dependencies play an important part in data normalization.

### **2. Query Optimization**

With the help of functional dependencies we are able to decide connectivity between tables and necessary attributes need to be projected to retrieve the required data from the tables. This helps in query optimization and improves performance.

### 3. Consistency of Data

Functional dependencies ensures consistency of data by removing any redundancies or inconsistencies that exist in data. Functional dependency ensures changes made in one attribute does not affect another set of attributes thus it maintains consistency of data in database.

### 4. Data Quality Improvement

Functional dependencies ensure that data in database to be accurate, complete, updated. This helps to improve overall quality of data, eliminates errors and inaccuracies that occur during data analysis and decision making, thus it helps in improving the quality of data in database.

**Closure of a set  $F$  of FDs** is the set  $F^+$  of all FDs that can be inferred from  $F$

### Data Modification Anomalies divided into three types:

- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

## **Normalization**

A large database defined as a single relation may result in data duplication. This repetition of data may result in:

- Making relations very large.
- It isn't easy to maintain and update data as it would involve searching many records in relation.
- Wastage and poor utilization of disk space and resources.
- The chances of errors and inconsistencies increases.

So to handle these problems, we should analyze and decompose relations with redundant data into smaller, simpler, and well-structured relations. **Normalization** can be defined as

- Normalization is the process of organizing the data in the database.
- Normalization is a process of decomposing the relations into relations with fewer attributes.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

Why do we need Normalization?

The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows.

## Types of Normal Forms

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:

|                           | 1NF                        | 2NF                                     | 3NF                                                   | 4NF                                                                      | 5NF                                                                                         |
|---------------------------|----------------------------|-----------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Decomposition of Relation | R                          | R <sub>11</sub><br>R <sub>12</sub>      | R <sub>21</sub><br>R <sub>22</sub><br>R <sub>23</sub> | R <sub>31</sub><br>R <sub>32</sub><br>R <sub>33</sub><br>R <sub>34</sub> | R <sub>41</sub><br>R <sub>42</sub><br>R <sub>43</sub><br>R <sub>44</sub><br>R <sub>45</sub> |
| Conditions                | Eliminate Repeating Groups | Eliminate Partial Functional Dependency | Eliminate Transitive Dependency                       | Eliminate Multi-values Dependency                                        | Eliminate Join Dependency                                                                   |

| Normal Form                | Description                                                                                                             |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#"><u>1NF</u></a> | A relation is in 1NF if it contains an atomic value.                                                                    |
| <a href="#"><u>2NF</u></a> | A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key. |
| <a href="#"><u>3NF</u></a> | A relation will be in 3NF if it is in 2NF and no transitive dependency exists.                                          |
| BCNF                       | A stronger definition of 3NF is known as Boyce Codd's normal form.                                                      |
| <a href="#"><u>4NF</u></a> | A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.                      |
| <a href="#"><u>5NF</u></a> | A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.             |

### Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

## Disadvantages of Normalization

- You cannot start building database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition lead to bad database design.

## First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that attribute of a table cannot hold multiple values. It must hold single-valued attribute.
- First normal form disallows multi-valued attribute, composite attribute, and their combinations.
- **Example:** Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP\_PHONE.

## EMPLOYEE table:

| EMP_ID | EMP_NAME | EMP_PHONE                 | EMP_STATE |
|--------|----------|---------------------------|-----------|
| 14     | John     | 7272826385,<br>9064738238 | UP        |
| 20     | Harry    | 8574783832                | Bihar     |
| 12     | Sam      | 7390372389,<br>8589830302 | Punjab    |

The decomposition of EMPLOYEE table into 1NF is below

| EMP_ID | EMP_NAME | EMP_PHONE  | EMP_STATE |
|--------|----------|------------|-----------|
| 14     | John     | 7272826385 | UP        |
| 14     | John     | 9064738238 | UP        |
| 20     | Harry    | 8574783832 | Bihar     |
| 12     | Sam      | 7390372389 | Punjab    |
| 12     | Sam      | 8589830302 | Punjab    |

## Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

**Example:** School can store data of teachers and subjects.  
In school, teacher can teach more than one subject.

## TEACHER table

| TEACHER_ID | SUBJECT   | TEACHER_AGE |
|------------|-----------|-------------|
| 25         | Chemistry | 30          |
| 25         | Biology   | 30          |
| 47         | English   | 35          |
| 83         | Math      | 38          |
| 83         | Computer  | 38          |

In the given table, non-prime attribute TEACHER\_AGE is dependent on TEACHER\_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

### TEACHER\_DETAIL table:

| TEACHER_ID | TEACHER_AGE |
|------------|-------------|
| 25         | 30          |
| 47         | 35          |
| 83         | 38          |

### TEACHER\_SUBJECT table:

| TEACHER_ID | SUBJECT   |
|------------|-----------|
| 25         | Chemistry |
| 25         | Biology   |
| 47         | English   |
| 83         | Math      |
| 83         | Computer  |

## Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.

- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency  $X \rightarrow Y$ .

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

**Example: EMPLOYEE\_DETAIL table:**

| EMP_ID | EMP_NAME  | EMP_ZIP | EMP_STATE | EMP_CITY |
|--------|-----------|---------|-----------|----------|
| 222    | Harry     | 201010  | UP        | Noida    |
| 333    | Stephan   | 02228   | US        | Boston   |
| 444    | Lan       | 60007   | US        | Chicago  |
| 555    | Katharine | 06389   | UK        | Norwich  |
| 666    | John      | 462007  | MP        | Bhopal   |

**Super key in the table above:**

1. {EMP\_ID}, {EMP\_ID, EMP\_NAME}, {EMP\_ID, EMP\_NAME, EMP\_ZIP}....so on

**Candidate key: {EMP\_ID}**

**Non-prime attributes:** In given table, all attributes except EMP\_ID are non-prime.

Here, EMP\_STATE & EMP\_CITY dependent on EMP\_ZIP and EMP\_ZIP dependent on EMP\_ID. The non-prime attributes (EMP\_STATE, EMP\_CITY) transitively dependent on super key(EMP\_ID). It violate rule of third normal form. That's why we need to move EMP\_CITY and EMP\_STATE to new <EMPLOYEE\_ZIP> table with EMP\_ZIP as Primary key.

### **EMPLOYEE table**

| EMP_ID | EMP_NAME  | EMP_ZIP |
|--------|-----------|---------|
| 222    | Harry     | 201010  |
| 333    | Stephan   | 02228   |
| 444    | Lan       | 60007   |
| 555    | Katharine | 06389   |
| 666    | John      | 462007  |

### **EMPLOYEE\_ZIP table**

| EMP_ZIP | EMP_STATE | EMP_CITY |
|---------|-----------|----------|
| 201010  | UP        | Noida    |
| 02228   | US        | Boston   |
| 60007   | US        | Chicago  |
| 06389   | UK        | Norwich  |
| 462007  | MP        | Bhopal   |

## Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency  $X \rightarrow Y$ , X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

**Example:** Let's assume there is a company where employees work in more than one department.

### EMPLOYEE table:

| EMP_ID | EMP_COUNTRY | EMP_DEPT   | DEPT_TYPE | EMP_DEPT_NO |
|--------|-------------|------------|-----------|-------------|
| 264    | India       | Designing  | D394      | 283         |
| 264    | India       | Testing    | D394      | 300         |
| 364    | UK          | Stores     | D283      | 232         |
| 364    | UK          | Developing | D283      | 549         |

**In above table Functional dependencies are as follows:**

1.  $EMP\_ID \rightarrow EMP\_COUNTRY$
2.  $EMP\_DEPT \rightarrow \{DEPT\_TYPE, EMP\_DEPT\_NO\}$

**Candidate key: {EMP-ID, EMP-DEPT}**

The table is not in BCNF because neither EMP\_DEPT nor EMP\_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

### EMP\_COUNTRY table:

| EMP_ID | EMP_COUNTRY |
|--------|-------------|
| 264    | India       |
| 264    | India       |

### EMP\_DEPT table:

| EMP_DEPT   | DEPT_TYPE | EMP_DEPT_NO |
|------------|-----------|-------------|
| Designing  | D394      | 283         |
| Testing    | D394      | 300         |
| Stores     | D283      | 232         |
| Developing | D283      | 549         |

### EMP\_DEPT\_MAPPING table:

| EMP_ID | EMP_DEPT |
|--------|----------|
| D394   | 283      |
| D394   | 300      |
| D283   | 232      |
| D283   | 549      |

### Functional dependencies:

1. EMP\_ID → EMP\_COUNTRY
2. EMP\_DEPT → {DEPT\_TYPE, EMP\_DEPT\_NO}

### Candidate keys:

**For the first table:** EMP\_ID

**For the second table:** EMP\_DEPT

**For the third table:** {EMP\_ID, EMP\_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

### **Fourth normal form (4NF)**

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency  $A \twoheadrightarrow B$ , if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

**Example:** STUDENT

| STU_ID | COURSE    | HOBBY   |
|--------|-----------|---------|
| 21     | Computer  | Dancing |
| 21     | Math      | Singing |
| 34     | Chemistry | Dancing |
| 74     | Biology   | Cricket |
| 59     | Physics   | Hockey  |

The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

In the STUDENT relation, a student with

STU\_ID **21** contains two courses **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-

valued dependency on STU\_ID, which leads to unnecessary repetition of data.

So to make above table into 4NF, we can decompose it into two tables:

### **STUDENT\_COURSE**

| STU_ID | COURSE    |
|--------|-----------|
| 21     | Computer  |
| 21     | Math      |
| 34     | Chemistry |
| 74     | Biology   |
| 59     | Physics   |

### **STUDENT\_HOBBY**

| STU_ID | HOBBY   |
|--------|---------|
| 21     | Dancing |
| 21     | Singing |
| 34     | Dancing |
| 74     | Cricket |
| 59     | Hockey  |

### **Fifth normal form (5NF)**

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

### Example

| SUBJECT   | LECTURER | SEMESTER   |
|-----------|----------|------------|
| Computer  | Anshika  | Semester 1 |
| Computer  | John     | Semester 1 |
| Math      | John     | Semester 1 |
| Math      | Akash    | Semester 2 |
| Chemistry | Praveen  | Semester 1 |

In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.

Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

## P1

| SEMESTER   | SUBJECT   |
|------------|-----------|
| Semester 1 | Computer  |
| Semester 1 | Math      |
| Semester 1 | Chemistry |
| Semester 2 | Math      |

## P2

| SUBJECT   | LECTURER |
|-----------|----------|
| Computer  | Anshika  |
| Computer  | John     |
| Math      | John     |
| Math      | Akash    |
| Chemistry | Praveen  |

## P3

| SEMSTER    | LECTURER |
|------------|----------|
| Semester 1 | Anshika  |
| Semester 1 | John     |
| Semester 1 | John     |
| Semester 2 | Akash    |
| Semester 1 | Praveen  |

## **SQL**

SQL stands for Structured Query Language. SQL is used to manipulate underlying relational databases that are queried by SQL like Oracle, MySQL, SQLite, etc.

### **Components of SQL**

#### **1. Keywords:**

Keywords are reserved or non-reserved words. SQL-reserved keywords are INTO, UPDATE, SELECT, DELETE, DROP, DESC, and ASC.

#### **2. Identifiers:**

The database objects, like function name, schema name, table name, etc., are named Identifiers.

#### **3. Clauses:**

The components of queries and SQL statements such as WHERE, GROUP BY, HAVING, and ORDER BY are formed by clauses.

#### **4. Expression:**

Either columns or scalar values and rows of data in SQL produced by EXPRESSION.

#### **5. Boolean Conditions:**

The boolean value TRUE or FALSE is the result of the Conditions, also called expressions. the effect of statements or queries limited by this condition.

#### **5. Queries:**

The data based on specific criteria is retrieved by the SQL statements. Queries are Statements that start with the SELECT clause because they retrieve data from the underlying database.

## **6. Statements:**

SQL statements may persistently affect schema and data or control transactions, program flow, connections, sessions, or diagnostics. SQL statements are such as INSERT, UPDATE, DROP, and DELETE statements since they modify the database structure or data.

## **Basic use of SQL:**

1. It modifies the database table and index structures.
2. It adds, updates, and deletes the rows of data.
3. Subsets of information from within the relational database management system are retrieved by it. The information from that can be used for the analytical application, transaction processing, and other applications which require communication with a relational database.

## SQL Data Types

Data types are used to represent the nature of the data that can be stored in the database table. For example, in a particular column of a table, if we want to store a string

type of data then we will have to declare a string data type of this column.

Data types mainly classified into three categories for every database.

- String Data types
- Numeric Data types
- Date and time Data types

## MySQL String Data Types

|                                   |                                                                                                                                                                                                                        |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CHAR(Size)</b>                 | It is used to specify a fixed length string that can contain numbers, letters, and special characters. Its size can be 0 to 255 characters. Default is 1.                                                              |
| <b>VARCHAR(Size)</b>              | It is used to specify a variable length string that can contain numbers, letters, and special characters. Its size can be from 0 to 65535 characters.                                                                  |
| <b>BINARY(Size)</b>               | It is equal to CHAR() but stores binary byte strings. Its size parameter specifies the column length in the bytes. Default is 1.                                                                                       |
| <b>VARBINARY(Size)</b>            | It is equal to VARCHAR() but stores binary byte strings. Its size parameter specifies the maximum column length in bytes.                                                                                              |
| <b>TEXT(Size)</b>                 | It holds a string that can contain a maximum length of 255 characters.                                                                                                                                                 |
| <b>TINYTEXT</b>                   | It holds a string with a maximum length of 255 characters.                                                                                                                                                             |
| <b>MEDIUMTEXT</b>                 | It holds a string with a maximum length of 16,777,215.                                                                                                                                                                 |
| <b>LONGTEXT</b>                   | It holds a string with a maximum length of 4,294,967,295 characters.                                                                                                                                                   |
| <b>ENUM(val1, val2, val3,...)</b> | It is used when a string object having only one value, chosen from a list of possible values. It contains 65535 values in an ENUM list. If you insert a value that is not in the list, a blank value will be inserted. |
| <b>SET(val1,val2,val3,...)</b>    | It is used to specify a string that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values at one time in a SET list.                                                          |

## MySQL Numeric Data Types

|                         |                                                                                                                                                                                                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BIT(Size)</b>        | It is used for a bit-value type. The number of bits per value is specified in size. Its size can be 1 to 64. The default value is 1.                                                                                                                                                                                           |
| <b>INT(size)</b>        | It is used for the integer value. Its signed range varies from -2147483648 to 2147483647 and unsigned range varies from 0 to 4294967295. The size parameter specifies the max display width that is 255.                                                                                                                       |
| <b>INTEGER(size)</b>    | It is equal to INT(size).                                                                                                                                                                                                                                                                                                      |
| <b>FLOAT(size, d)</b>   | It is used to specify a floating point number. Its size parameter specifies the total number of digits. The number of digits after the decimal point is specified by <b>d</b> parameter.                                                                                                                                       |
| <b>FLOAT(p)</b>         | It is used to specify a floating point number. MySQL used p parameter to determine whether to use FLOAT or DOUBLE. If p is between 0 to 24, the data type becomes FLOAT (). If p is from 25 to 53, the data type becomes DOUBLE().                                                                                             |
| <b>DOUBLE(size, d)</b>  | It is a normal size floating point number. Its size parameter specifies the total number of digits. The number of digits after the decimal is specified by d parameter.                                                                                                                                                        |
| <b>DECIMAL(size, d)</b> | It is used to specify a fixed point number. Its size parameter specifies the total number of digits. The number of digits after the decimal parameter is specified by <b>d</b> parameter. The maximum value for the size is 65, and the default value is 10. The maximum value for <b>d</b> is 30, and the default value is 0. |
| <b>DEC(size, d)</b>     | It is equal to DECIMAL(size, d).                                                                                                                                                                                                                                                                                               |
| <b>BOOL</b>             | It is used to specify Boolean values true and false. Zero is considered as false, and nonzero values are considered as true.                                                                                                                                                                                                   |

## MySQL Date and Time Data Types

|                       |                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DATE</b>           | It is used to specify date format YYYY-MM-DD. Its supported range is from '1000-01-01' to '9999-12-31'.                                                                                                                                                   |
| <b>DATETIME(fsp)</b>  | It is used to specify date and time combination. Its format is YYYY-MM-DD hh:mm:ss. Its supported range is from '1000-01-01 00:00:00' to 9999-12-31 23:59:59'.                                                                                            |
| <b>TIMESTAMP(fsp)</b> | It is used to specify the timestamp. Its value is stored as the number of seconds since the Unix epoch('1970-01-01 00:00:00' UTC). Its format is YYYY-MM-DD hh:mm:ss. Its supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. |

|                  |                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>TIME(fsp)</b> | It is used to specify the time format. Its format is hh:mm:ss. Its supported range is from '838:59:59' to '838:59:59' |
| <b>YEAR</b>      | It is used to specify a year in four-digit format. Values allowed in four digit format from 1901 to 2155, and 0000.   |

## **Basic Queries of SQL**

### **1.INSERT INTO Statement**

This SQL statement inserts the data or records in the existing table of the SQL database. This statement can easily insert single and multiple records in a single query statement.

#### **Syntax of insert a single record:**

1. **INSERT INTO** table\_name
2. (
3. column\_name1,
4. column\_name2, ...,
5. column\_nameN
6. )
7. **VALUES**
8. (value\_1,
9. value\_2, .....,
10. value\_N
11. );

#### **Example of insert a single record:**

1. **INSERT INTO** Employee\_details
2. (
3. Emp\_ID,
4. First\_name,
5. Last\_name,

6. Salary,
7. City
8. )
9. **VALUES**
10. (101,
11. Akhil,
12. Sharma,
13. 40000,
14. Bangalore
15. );

This example inserts **101** in the first column, **Akhil** in the second column, **Sharma** in the third column, **40000** in the fourth column, and **Bangalore** in the last column of the table **Employee\_details**.

## 2.UPDATE Statement

This SQL statement changes or modifies the stored data in the SQL database.

### Syntax of UPDATE Statement:

1. **UPDATE** table\_name
2. **SET** column\_name1 = new\_value\_1, column\_name2 = new\_value\_2, ..., column\_nameN = new\_value\_N
3. [ **WHERE** CONDITION ];

### Example of UPDATE Statement:

1. **UPDATE** Employee\_details
2. **SET** Salary = 100000
3. **WHERE** Emp\_ID = 10;

This example changes the **Salary** of those employees of the **Employee\_details** table whose **Emp\_ID** is **10** in the table.

## 3. DELETE Statement

This SQL statement deletes the stored data from the SQL database.

### Syntax of DELETE Statement:

1. **DELETE FROM** table\_name
2. [ **WHERE** CONDITION ];

### Example of DELETE Statement:

1. **DELETE FROM** Employee\_details
2. **WHERE** First\_Name = 'Sumit';

This example deletes the record of those employees from the **Employee\_details** table whose **First\_Name** is **Sumit** in the table.

This example inserts **101** in the first column, **Akhil** in the second column, **Sharma** in the third column, **40000** in the fourth column, and **Bangalore** in the last column of the table **Employee\_details**.

## 4.View in SQL

A view is a SQL statement stored in the database with a name linked to it. It can store all table rows or only a few selected rows from the table. The user can create a view in SQL using single or multiple tables. The users create a view so that the data stored in a specific table can be represented as virtual tables. It also enables the administrator to restrict access to the data so that the user can only view or edit exactly the particular element of the table they want to without changing the rest.

## Creating Views

If the user wants to create a view in the database, then the user can do so by implementing CREATE VIEW statement. The user can use a single or multiple tables to create views. Mainly the views are created by the database administrator.

### Syntax to Implement VIEW

1. **CREATE VIEW** view\_name **AS**
2. **SELECT** column1, column2.....
3. **FROM** table\_name
4. **WHERE** [condition];

We have used a single table in the above syntax, but the user can include multiple tables in the SELECT statement using the same syntax used in any other SQL SELECT query.

## **Query Processing in DBMS**

Query Processing is the activity performed in extracting data from the database. In query processing, it takes various steps for fetching the data from the database.

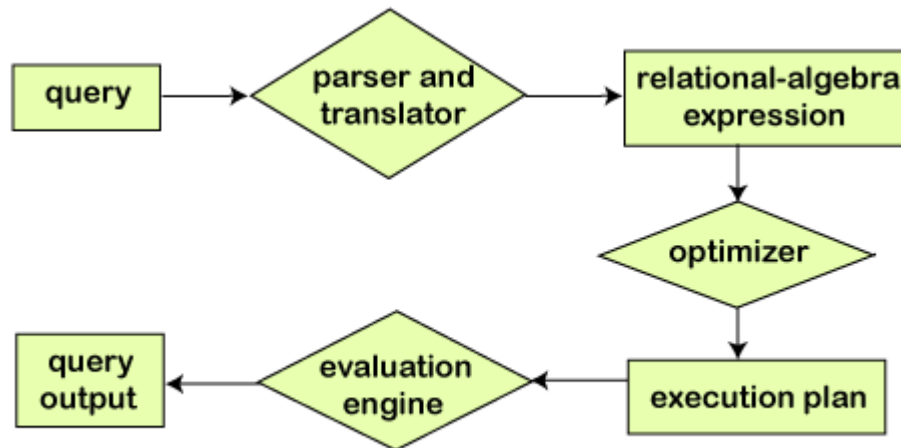
**The steps involved are:**

1. Parsing and translation
2. Optimization
3. Evaluation

### **1. Parsing and Translation**

SQL or Structured Query Language is the best suitable choice for humans. But, it is not perfectly suitable for the internal representation of query to system. Relational algebra is well suited for internal representation of a query. The translation process in query processing is similar to the parser of a query. When a user executes any query, for generating internal form of the query, the parser in system checks the syntax of query, verifies the name of relation in the database, the tuple, and finally the required attribute value. The parser creates a tree of the query, known as 'parse-tree.' Further, translate it into form of relational algebra. With this, it even replaces all the use of the views when used in the query.

Thus, we can understand the working of a query processing in the below-described diagram:



Suppose a user executes a query. In SQL, a user wants to fetch records of employees whose salary is greater than or equal to 10000. For this, following query is executed.

**select emp\_name from Employee where salary>10000;**

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{\text{salary} > 10000} (\pi_{\text{salary}} (\text{Employee}))$
- $\pi_{\text{salary}} (\sigma_{\text{salary} > 10000} (\text{Employee}))$

After translating the given query, we can execute each relational algebra operation by using different

algorithms. So, in this way, a query processing begins its working.

## **2.Optimization**

- The cost of query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as **Query Optimization**.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

## **3.Evaluation**

With addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in evaluation plan may refer to the algorithms to be used for particular index or the specific operations.
- Such relational algebra with annotations is known as **Evaluation Primitives**. Evaluation primitives carry instructions needed for evaluation of operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also known as **query execution plan**.
- A **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of query.

### **Concurrency Control**

Concurrency Control is the management procedure that is required for controlling concurrent execution of the operations that take place on a database.

But before knowing about concurrency control, we should know about concurrent execution.

### **Concurrent Execution in DBMS**

- In a multi-user system, multiple users can access and use the same database at one time, which is known as concurrent execution of the database. It means that the same database is executed simultaneously on a multi-user system by different users.
- While working on database transactions, multiple users can perform different operations and in that case concurrent execution of database is performed.
- The thing is that the simultaneous execution that is performed should be done in a manner that no operation should affect the other executing operations, thus maintaining the consistency of the database. Thus, on making the concurrent execution of the transaction operations, there occur several challenging problems that need to be solved.

### **Problems with Concurrent Execution**

In a database transaction, the two main operations are **READ** and **WRITE** operations. So, there is a need to manage these two operations in concurrent execution of the transactions as if these operations are not performed in an interleaved manner, the data may become

inconsistent. So, the following problems occur with the Concurrent Execution of the operations:

### **Problem 1: Lost Update Problems (W - W Conflict)**

The problem occurs when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.

**For example:**

**Consider the below diagram where two transactions  $T_x$  and  $T_y$ , are performed on the same account A where the balance of account A is \$300.**

| Time  | $T_x$        | $T_y$         |
|-------|--------------|---------------|
| $t_1$ | READ (A)     | —             |
| $t_2$ | $A = A - 50$ |               |
| $t_3$ | —            | READ (A)      |
| $t_4$ | —            | $A = A + 100$ |
| $t_5$ | —            | —             |
| $t_6$ | WRITE (A)    | —             |
| $t_7$ |              | WRITE (A)     |

**LOST UPDATE PROBLEM**

- At time  $t_1$ , transaction  $T_x$  reads the value of account A, i.e., \$300 (only read).
- At time  $t_2$ , transaction  $T_x$  deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
- Alternately, at time  $t_3$ , transaction  $T_y$  reads the value of account A that will be \$300 only because  $T_x$  didn't update the value yet.
- At time  $t_4$ , transaction  $T_y$  adds \$100 to account A that becomes \$400 (only added but not updated/write).
- At time  $t_6$ , transaction  $T_x$  writes the value of account A that will be updated as \$250 only, as  $T_y$  didn't update the value yet.
- Similarly, at time  $t_7$ , transaction  $T_y$  writes the values of account A, so it will write as done at time  $t_4$  that will be \$400. It means the value written by  $T_x$  is lost, i.e., \$250 is lost.

Hence data becomes incorrect, and database sets to inconsistent.

## **2.Dirty Read Problems (W-R Conflict)**

The dirty read problem occurs when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another

transaction. There comes the Read-Write Conflict between both transactions.

**For example:**

**Consider two transactions  $T_x$  and  $T_y$  in the below diagram performing read/write operations on account A where the available balance in account A is \$300:**

| Time  | $T_x$                   | $T_y$    |
|-------|-------------------------|----------|
| $t_1$ | READ (A)                | —        |
| $t_2$ | $A = A + 50$            | —        |
| $t_3$ | WRITE (A)               | —        |
| $t_4$ | —                       | READ (A) |
| $t_5$ | SERVER DOWN<br>ROLLBACK | —        |

**DIRTY READ PROBLEM**

- At time  $t_1$ , transaction  $T_x$  reads the value of account A, i.e., \$300.
- At time  $t_2$ , transaction  $T_x$  adds \$50 to account A that becomes \$350.
- At time  $t_3$ , transaction  $T_x$  writes the updated value in account A, i.e., \$350.
- Then at time  $t_4$ , transaction  $T_y$  reads account A that will be read as \$350.

- Then at time  $t_5$ , transaction  $T_x$  rolls back due to server problem, and the value changes back to \$300 (as initially).
- But the value for account A remains \$350 for transaction  $T_y$  as committed, which is the dirty read and therefore known as the Dirty Read Problem.

Thus, in order to maintain consistency in the database and avoid such problems that take place in concurrent execution, management is needed, and that is where the concept of Concurrency Control comes into role.

### **Concurrency Control**

Concurrency Control is required for controlling and managing the concurrent execution of database operations and thus avoiding the inconsistencies in the database. Thus, for maintaining the concurrency of the database, we have the concurrency control protocols.

### **Concurrency Control Protocols**

The concurrency control protocols ensure the atomicity, consistency, isolation, durability and serializability of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:

- Lock Based Concurrency Control Protocol
- Time Stamp Concurrency Control Protocol

- Validation Based Concurrency Control Protocol

## **1. Lock-Based Protocol**

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

### **a) Shared lock:**

- It is also known as a Read-only lock. In a shared lock, data item can be only read by transaction.
- It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.

### **b) Exclusive lock:**

- In the exclusive lock, the data item can be both reads as well as written by the transaction.
- This lock is exclusive and in this lock, multiple transactions do not modify the same data simultaneously.

## **2. Timestamp Ordering Protocol**

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is ascending order of the transaction creation.
- The priority of older transaction is higher that's why it executes first. To determine timestamp of the

transaction, this protocol uses system time or logical counter.

- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

### Advantages of TO protocol

- This protocol ensures serializability since the precedence graph is as follows:



**Image:** Precedence Graph for TS ordering

- This protocol ensures freedom from deadlock that means no transaction ever waits.

### **3.Validation Based Protocol**

Validation phase is also known as optimistic concurrency control technique. In the validation based protocol, the transaction is executed in the following three phases:

1. **Read phase:** In this phase, transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.
2. **Validation phase:** In this phase, the temporary variable value will be validated against the actual data to see if it violates the serializability.
3. **Write phase:** If validation of transaction is validated, then temporary results are written to the database or system, otherwise the transaction is rolled back.

Here each phase has the following different timestamps:

**Start( $T_i$ ):** It contains time when  $T_i$  started its execution.

**Validation ( $T_i$ ):** It contains the time when  $T_i$  finishes its read phase and starts its validation phase.

**Finish( $T_i$ ):** It contains time when  $T_i$  finishes write phase.

- The serializability is determined during the validation process. It can't be decided in advance.
- While executing transaction, it ensures a greater degree of concurrency and also less number of conflicts.
- Thus it contains transactions which have less number of rollbacks.